# AI-BASED IMAGE COMPRESSION

SAMUELE PONZIN

# Traditional image compression

Data compression is commonly defined with the goal of designing codes for a given discrete data ensemble with minimal entropy (rate).

The solution relies heavily on knowledge of the probabilistic structure of the data (probabilistic source modeling).

Since all codes have finite entropy, continuous-valued data must be quantized to a finite of set discrete values (quantization), which introduces error (distortion).

# Transform block coding

Any source can be optimally compressed in theory using vector quantization (VQ).

VQ quickly becomes computationally infeasible for sources with dimensionality: mainly because the codebook of reproduction vectors, as well as the computational complexity of the search for the best reproduction of the source vector grow exponentially with the number of dimensions.

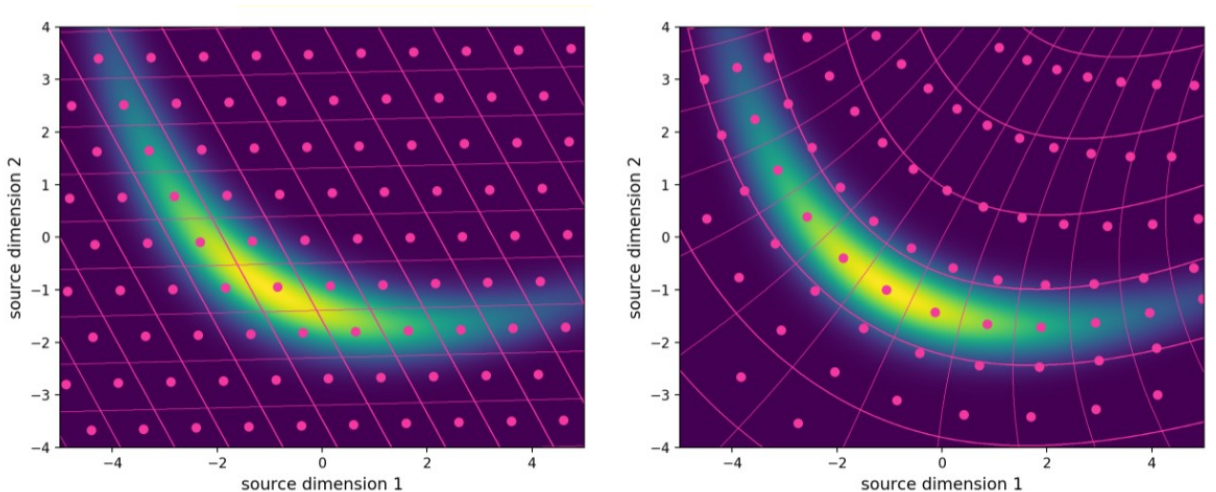
Linear (orthogonal) transforms are applied to move in a more suitable continuous-valued representation (latent space), **quantizing and coding each dimension independently**.

In this way we can turn a source with memory to a memoryless source, composed by separable i.i.d. random variables (actually uncorrelated, not independent).

# Non linear transform coding

Linear transformation, quantization and entropy coding are **separately optimized** (often just manual parameters adjustment).

The problem with linear transforms is that the joint statistics of linear filter responses exhibit strong higher order dependencies. These may be significantly reduced through the use of **joint local nonlinear gain control operations**, inspired by models of visual neurons.




J. Ballé et al., "Nonlinear Transform Coding," in IEEE Journal of Selected Topics in Signal Processing, vol. 15, no. 2, pp. 339-353, Feb. 2021, doi: 10.1109/JSTSP.2020.3034501.

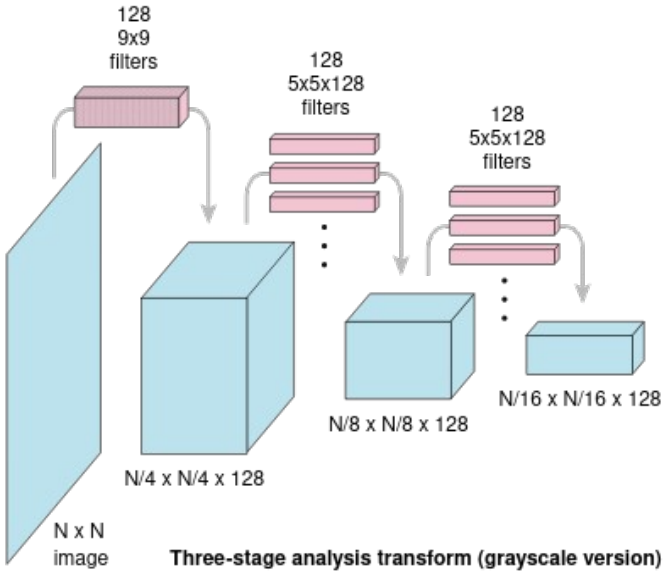
# End to End Optimal Image Compression (BLS2017)

A framework for end-to-end optimization of an image compression model based on nonlinear transforms.

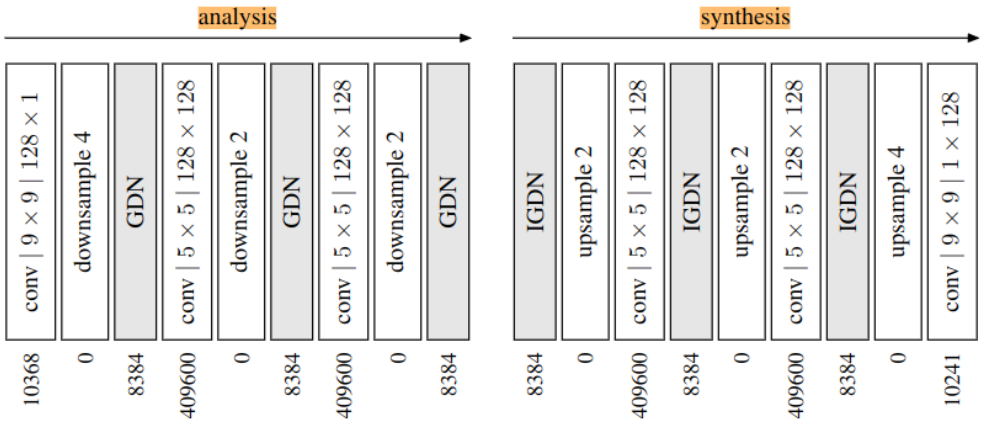
As in many other optimized representations the filters in this first stage are localized and oriented and the representation is overcomplete. The ability of the model to better represent features and contours with continuously varying orientation, position and scale.

The joint nonlinearity is chosen to implement a form of local gain control, inspired by those used to model biological neurons.

# BLS2017 - Architecture



Three-stage analysis transform (grayscale version)



# BLS2017 – Generative Divisive Normalization

Inspired by models of neurons in biological visual systems and has proven effective in Gaussianizing image densities. Spatially adaptive and can be highly nonlinear, it is used in NNs as nonlinear activation + normalization.

Better for image compression w.r.t. batch normalization, where the rescaling factor is identical across all spatial locations (not a local grain control).

More efficient than ReLU activation, achieves better performance with fewer processing stages.

GDN

$$u_i^{(k+1)}(m,n) = \frac{w_i^{(k)}(m,n)}{\left(\beta_{k,i} + \sum_j \gamma_{k,ij}\left(w_j^{(k)}(m,n)\right)^2\right)^{\frac{1}{2}}}$$

IGDN

$$\hat{w}_i^{(k)}(m,n) = \hat{u}_i^{(k)}(m,n) \cdot \left(\hat{\beta}_{k,i} + \sum_j \hat{\gamma}_{k,ij}\left(\hat{u}_j^{(k)}(m,n)\right)^2\right)^{\frac{1}{2}}$$

# BLS2017 – Quantization

The cascaded analysis transformation is followed by uniform scalar quantization **which effectively implements a parametric form of vector quantization in the original image space**.

A fixed uniform scalar quantizer in the code space rather than attempting optimal quantization directly in the image space, aiming to have the nonlinear transformations warp the space in an appropriate way, implementing a parametric form of vector quantization.

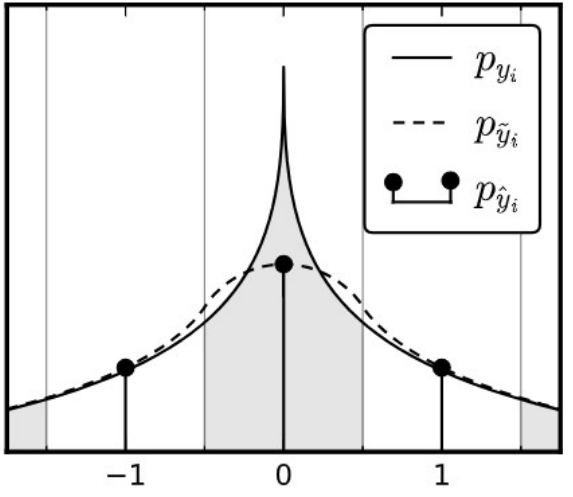
# BLS2017 – Quantization

To allow optimization via stochastic gradient, the quantizer is replaced with an additive i.i.d. uniform noise source Δy, which has the same width as the quantization bins.

**Properties**

- the density function of $\tilde{y} = y + \Delta y$ is a continuous relaxation of the probability mass function of the quantized $\hat{y} = Q(y)$, which implies that the differential entropy of $\tilde{y}$ can be used as an approximation of the entropy of $\hat{y}$.

- independent uniform noise approximates quantization error in terms of its marginal moments, and is frequently used as a model of quantization error. It is possible to use the same approximation for the distortion.




Ballé, Johannes & Laparra, Valero & Simoncelli, Eero. (2016). End-to-end Optimized Image Compression.

# BLS2017 – Loss function

Our objective is to minimize a weighted sum of the rate and distortion, R + λD, over the parameters of the analysis transform; the synthesis transform and the entropy code.

λ governs the trade-off between the two terms.

$$L(\boldsymbol{\theta}, \boldsymbol{\phi}) = \mathbb{E}_{\boldsymbol{x}, \Delta \boldsymbol{y}} \left[ - \sum_i \log_2 p_{\tilde{y}_i}(g_a(\boldsymbol{x}; \boldsymbol{\phi}) + \Delta \boldsymbol{y}; \boldsymbol{\psi}^{(i)}) \quad + \lambda \, d\Big( g_p\big(g_s(g_a(\boldsymbol{x}; \boldsymbol{\phi}) + \Delta \boldsymbol{y}; \boldsymbol{\theta})\big), g_p(\boldsymbol{x}) \Big) \right]$$

The density function of $\tilde{y} = y + \Delta y$ is a continuous relaxation of the probability mass function of the latent vector, that mimics the quantization during training.

The vector $\psi^{(i)}$ parameterizes the piecewise linear approximation of the marginals $p_{\tilde{y}_i}$.

# BLS2017 – Example

Lambda: 1
N. of filters: 128

Distortion (MSE): 26.6825
Rate (BPP): 1.9695

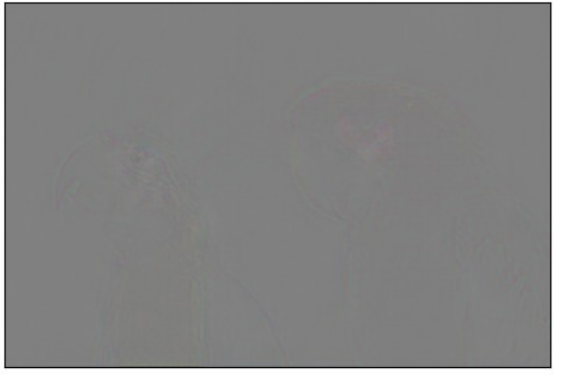
Original Image



Reconstructed image



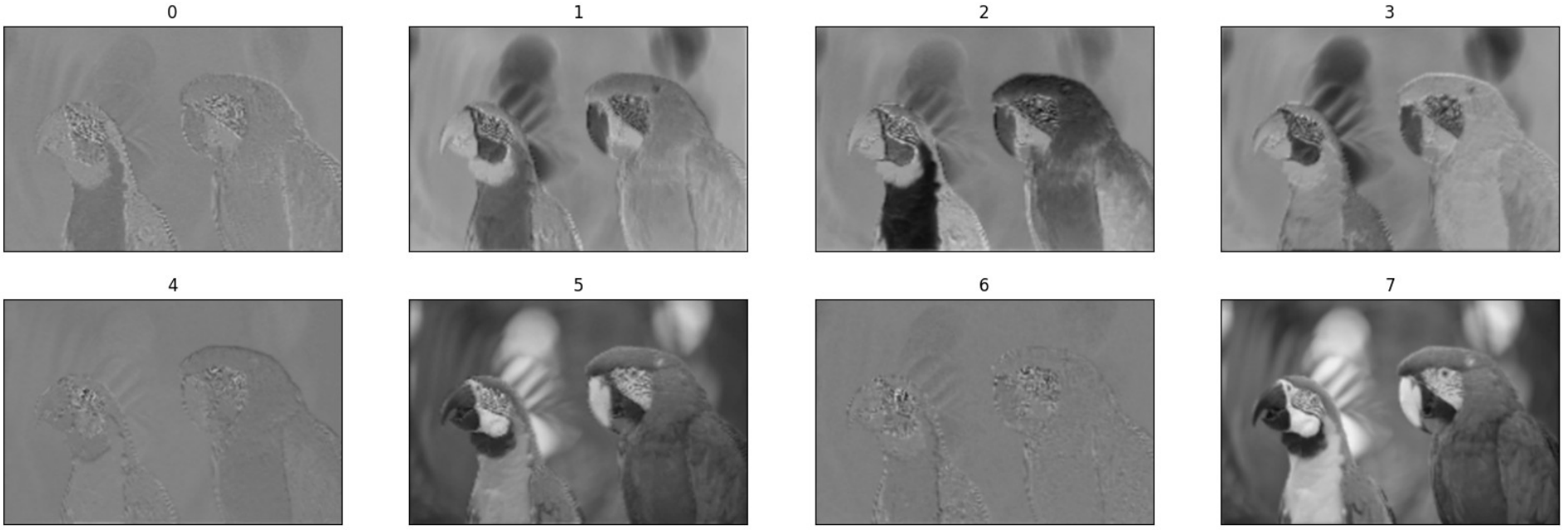
Error image



# BLS2017 – Feature maps (1st analysis step)



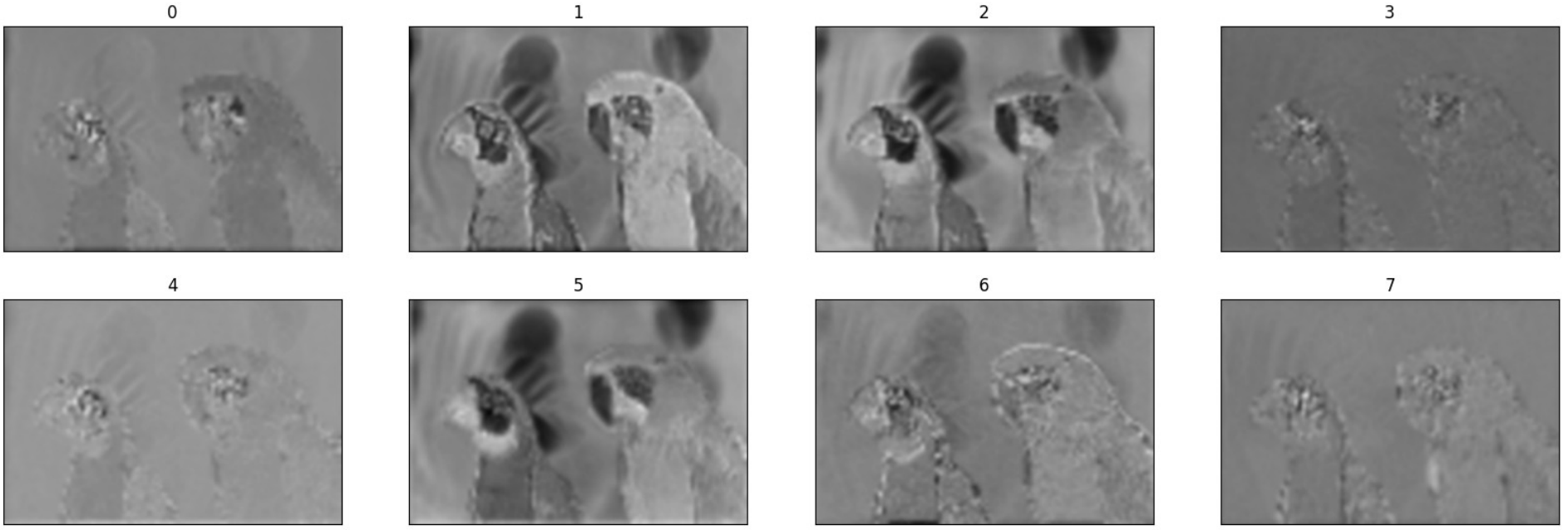
# BLS2017 – Feature maps (1st analysis step)



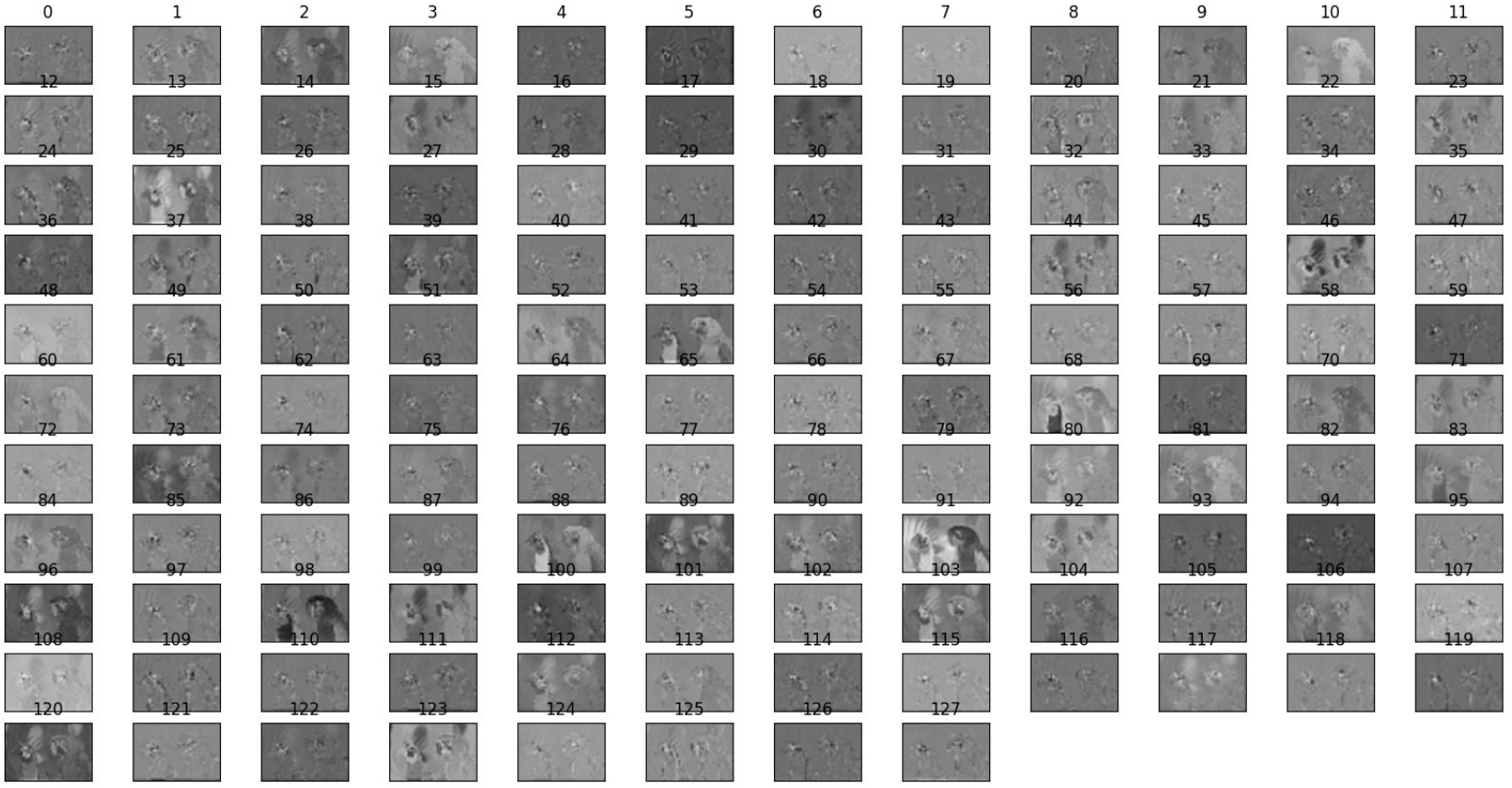
# BLS2017 – Feature maps (2nd analysis step)



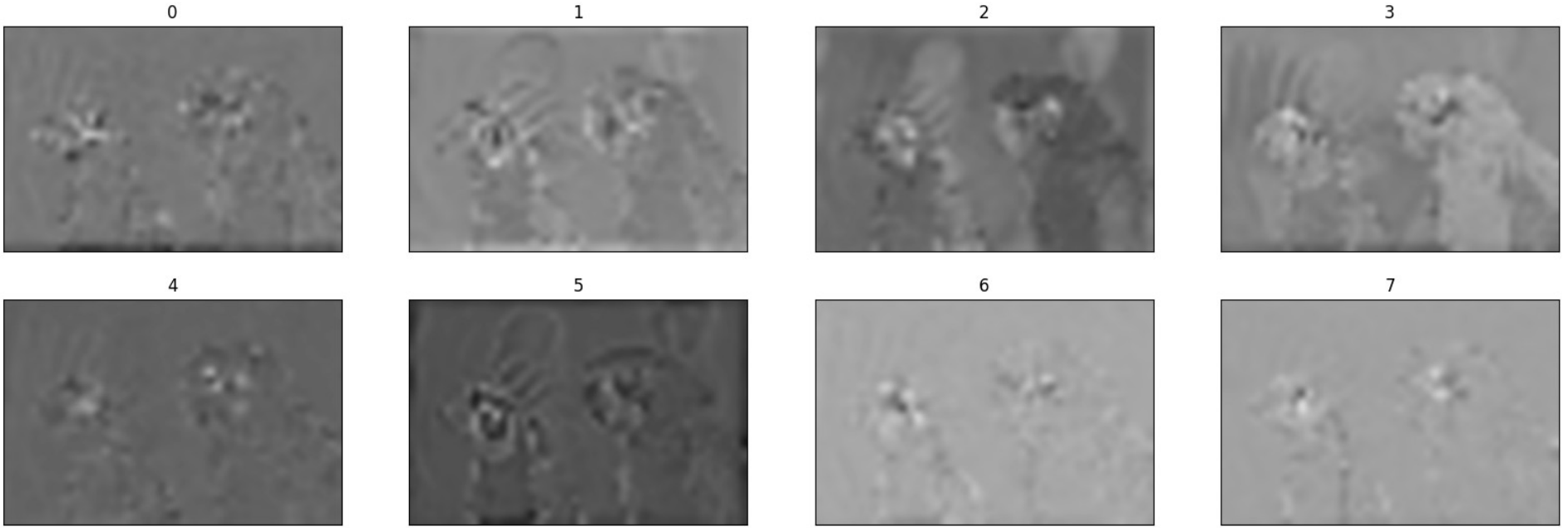
# BLS2017 – Feature maps (2nd analysis step)



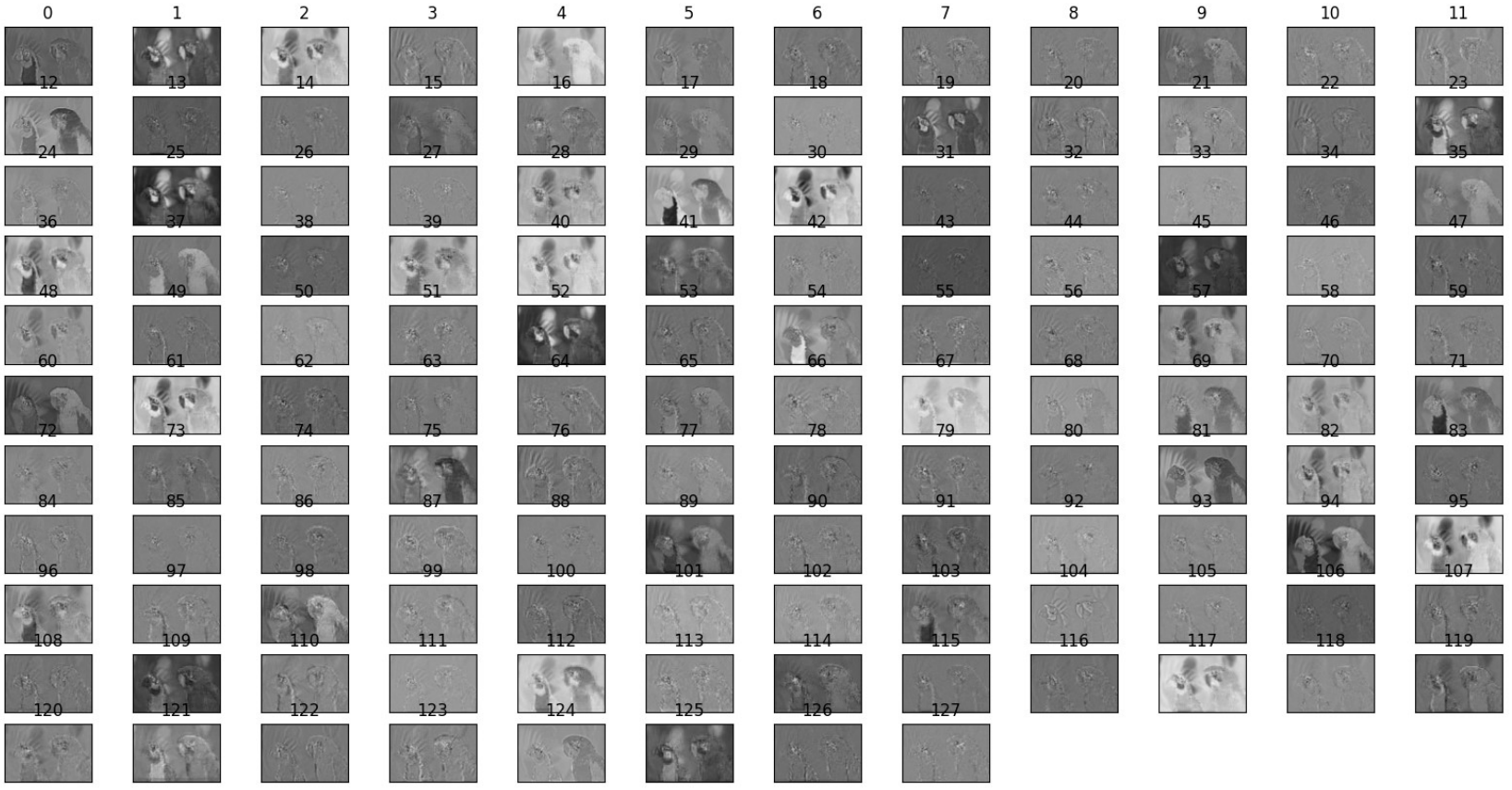
# BLS2017 – Feature maps (3rd analysis step)



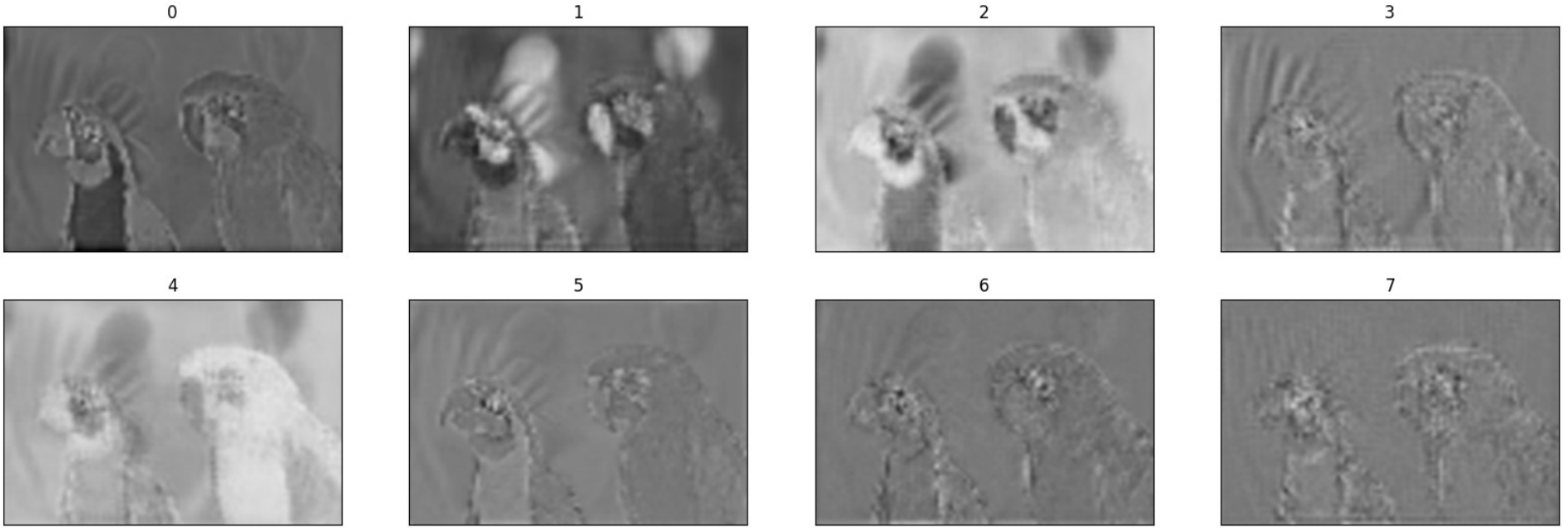
# BLS2017 – Feature maps (3rd analysis step)



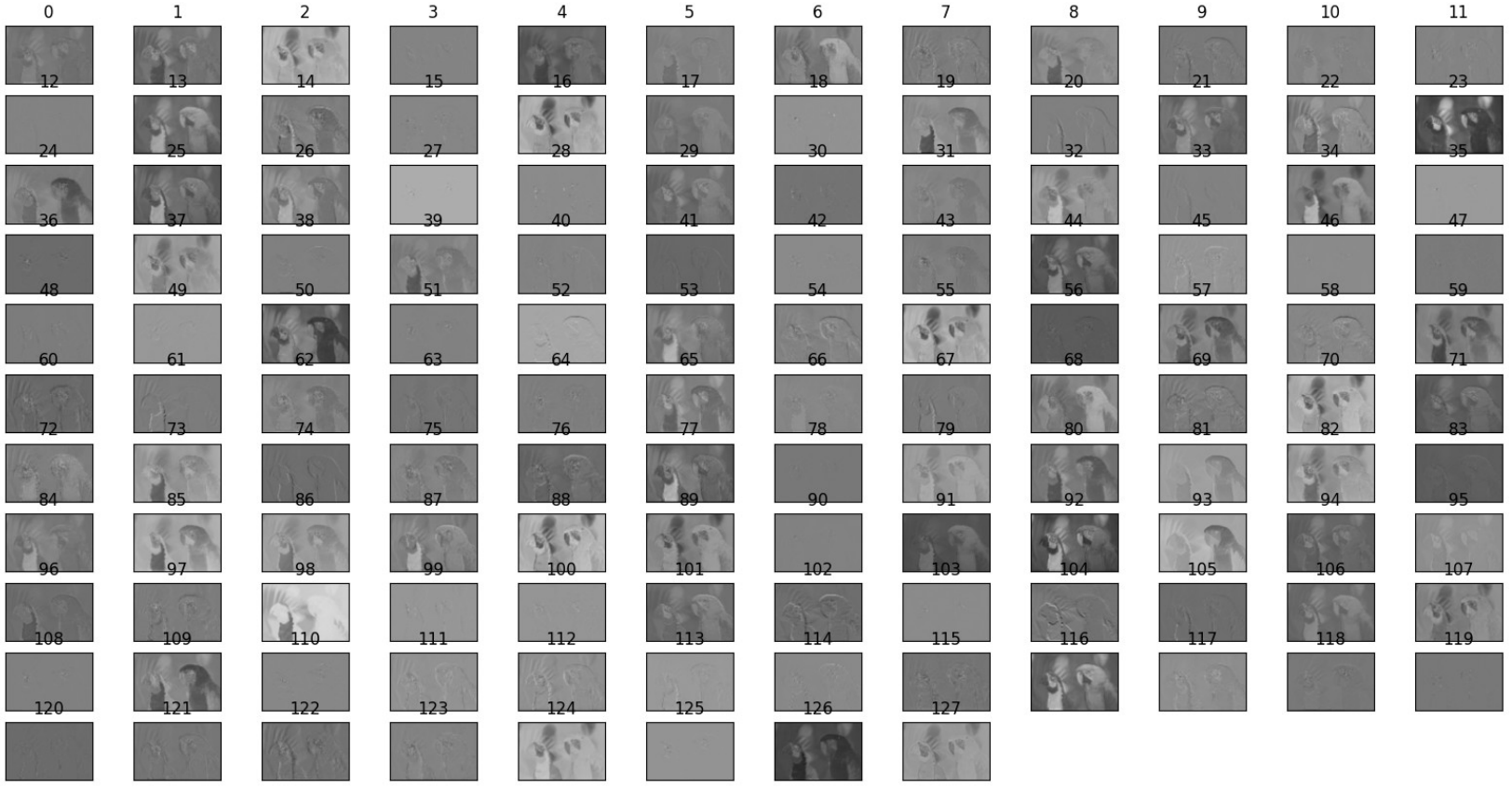
# BLS2017 – Feature maps (1st synthesis step)



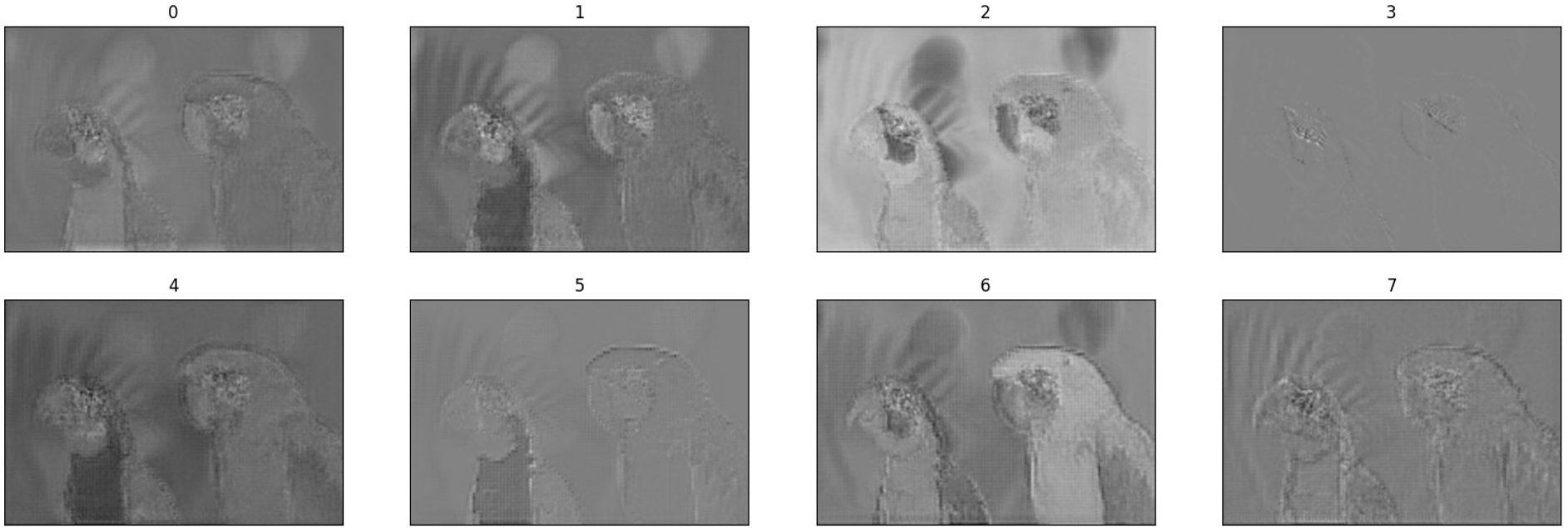
# BLS2017 – Feature maps (1st synthesis step)



# BLS2017 – Feature maps (2nd synthesis step)



# BLS2017 – Feature maps (2nd synthesis step)



# BLS2017 - Results

bls2017_lambda_000_001



bls2017_lambda_000_010



bls2017_lambda_000_100



bls2017_lambda_001_000



bls2017_lambda_010_000



bls2017_lambda_100_000



# BLS2017 – JPEG comparison

Learned nonlinear compression preserves the smoothness of contours and sharpness of many of the edges, giving them a natural appearance.

JPEG and JPEG 2000 images exhibit artifacts that are common to all linear transform coding methods: independent scalar quantization of the transform coefficients causes imbalances in the linear combinations of the basis functions and leads to blocking, aliasing and ringing artifacts that reflect the underlying basis functions.

As bit rate is reduced, JPEG and JPEG 2000 degrade their approximation of the original image by coarsening the precision of the coefficients of linear basis functions, thus exposing the visual appearance of those basis functions.

bls2017_lambda_000_001





# BLS2017 - Results

bls2017_lambda_000_001




bls2017_lambda_000_010




bls2017_lambda_000_100




bls2017_lambda_001_000




bls2017_lambda_010_000




bls2017_lambda_100_000



# BLS2017 - Results

bls2017_lambda_000_001



bls2017_lambda_000_010



bls2017_lambda_000_100



bls2017_lambda_001_000



bls2017_lambda_010_000



bls2017_lambda_100_000



# BLS2017 - Results

bls2017_lambda_000_001



bls2017_lambda_000_010



bls2017_lambda_000_100



bls2017_lambda_001_000



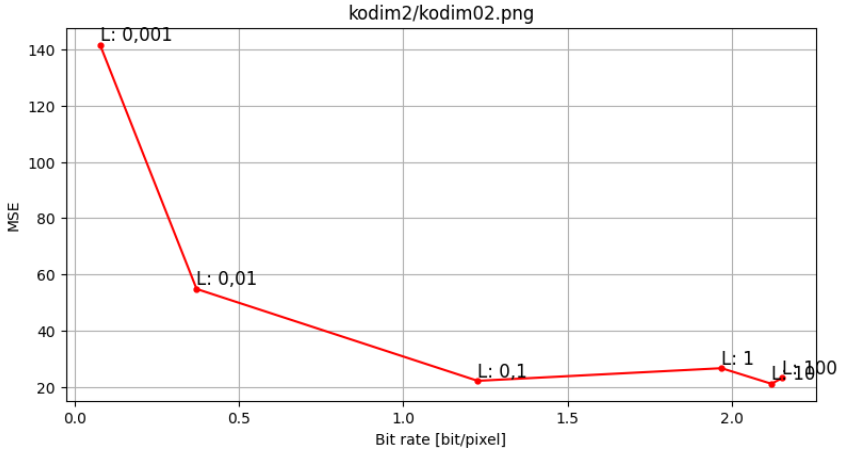
bls2017_lambda_010_000

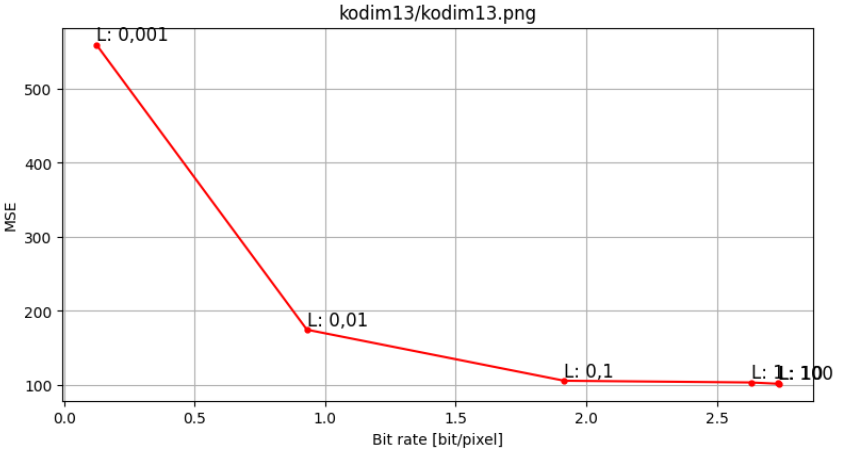


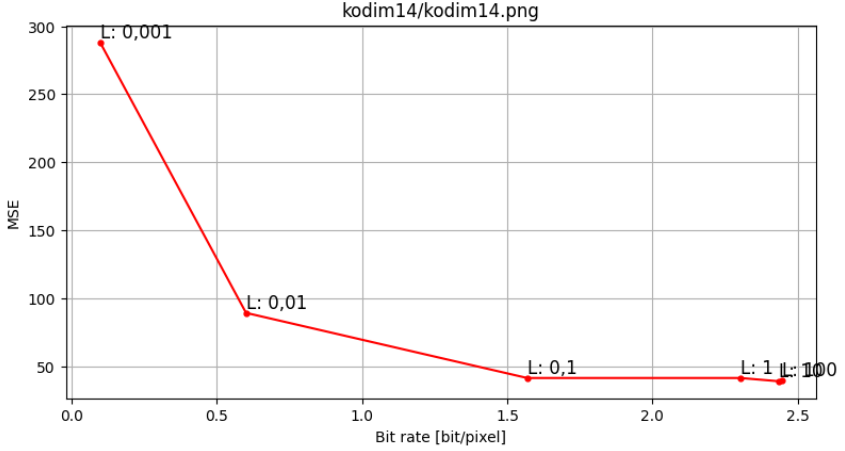
bls2017_lambda_100_000

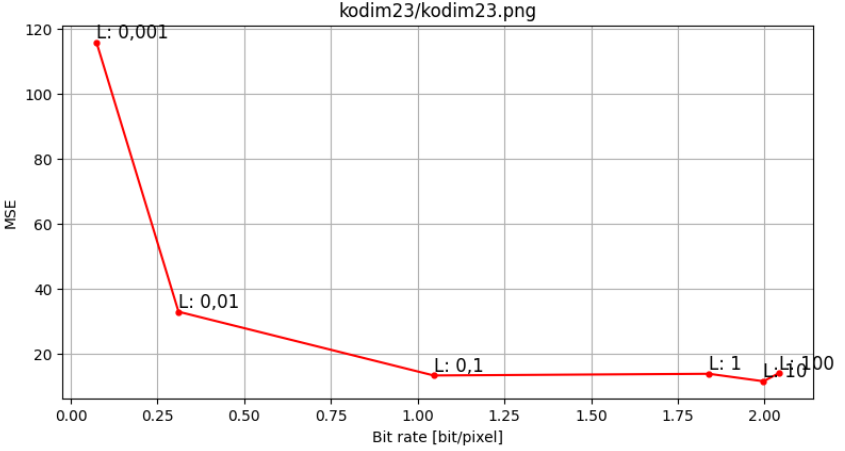


# BLS2017 - Results



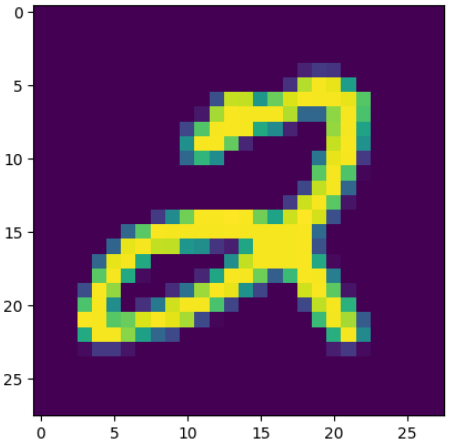



# MNIST compression

**Analysis transform**
1) 2DConv (5x5), 20 filters, stride 2, leaky ReLU
2) 2DConv (5x5), 50 filters, stride 2, leaky ReLU
3) Vectorization: (7,7,50) → (2450,)
4) Dense, 500 output units, leaky ReLU
5) Dense, 10 output units, leaky ReLU

**Synthesis transform**
1) Dense, 500 output units, leaky ReLU
2) Dense, 2450 output units, leaky ReLU
3) Reshape: (2450,) → (7,7,50)
4) 2DConvTranspose (5x5), 20 filters, stride 2, leaky ReLU
5) 2DConvTranspose (5x5), 1 filter, stride 2, leaky ReLU

Input image (28x28x1)



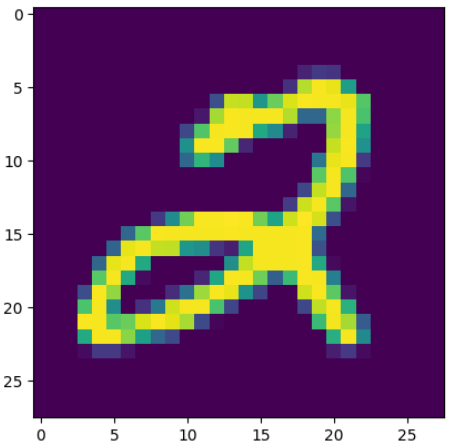
# MNIST compression

**Dataset:** MNIST digits
- training set cardinality: 60,000
- testing set cardinality: 10,000

**Training**
- Distortion measure: MAE
- Optimizer: Adam
- Learning Rate: 0.001
- Epochs: 15

- lambda: 2000/500/300
- latent dimension: 10

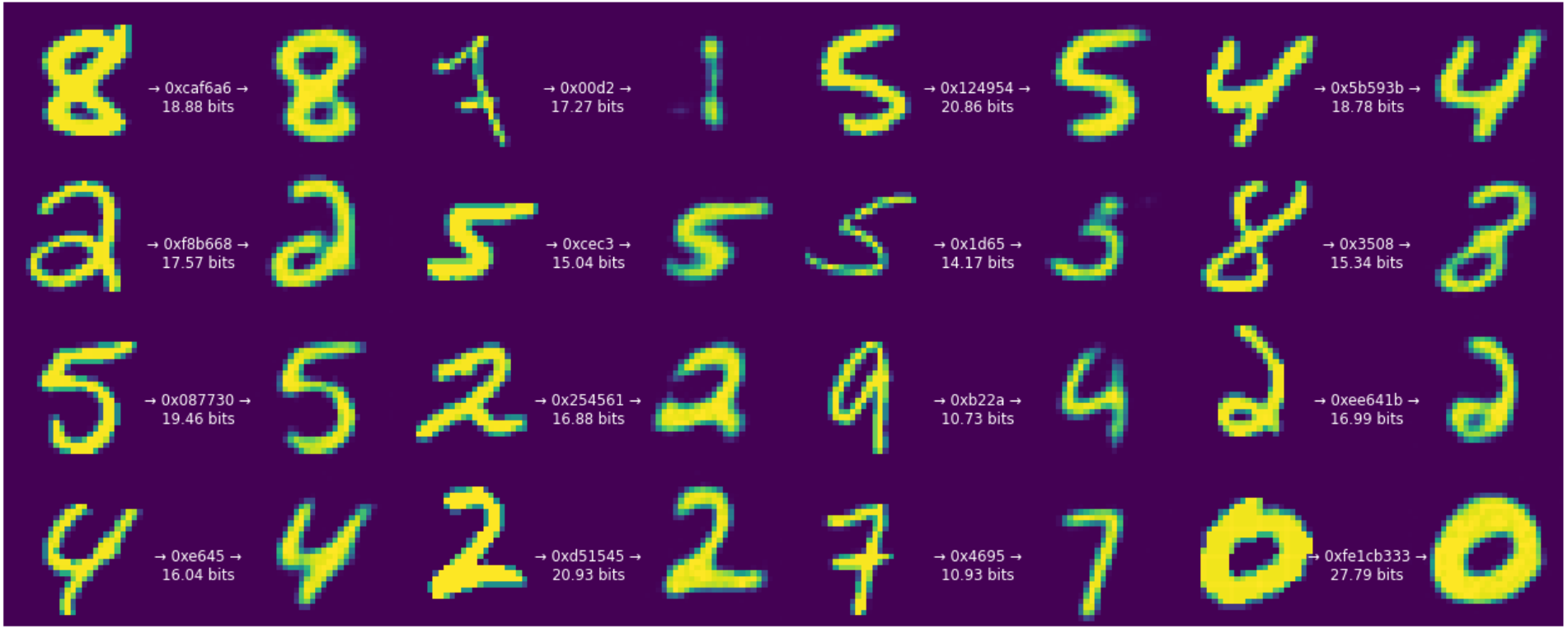
Input image (28x28x1)



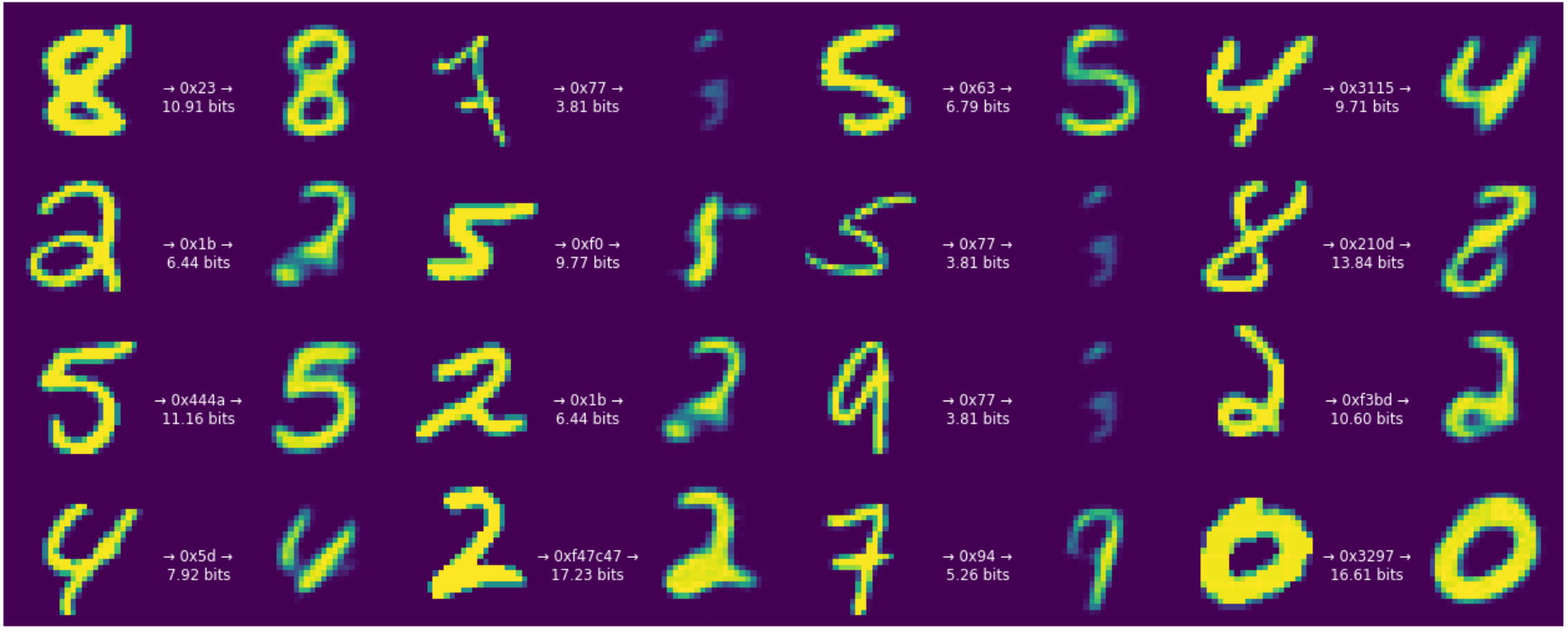
# MNIST compression (λ = 2000)



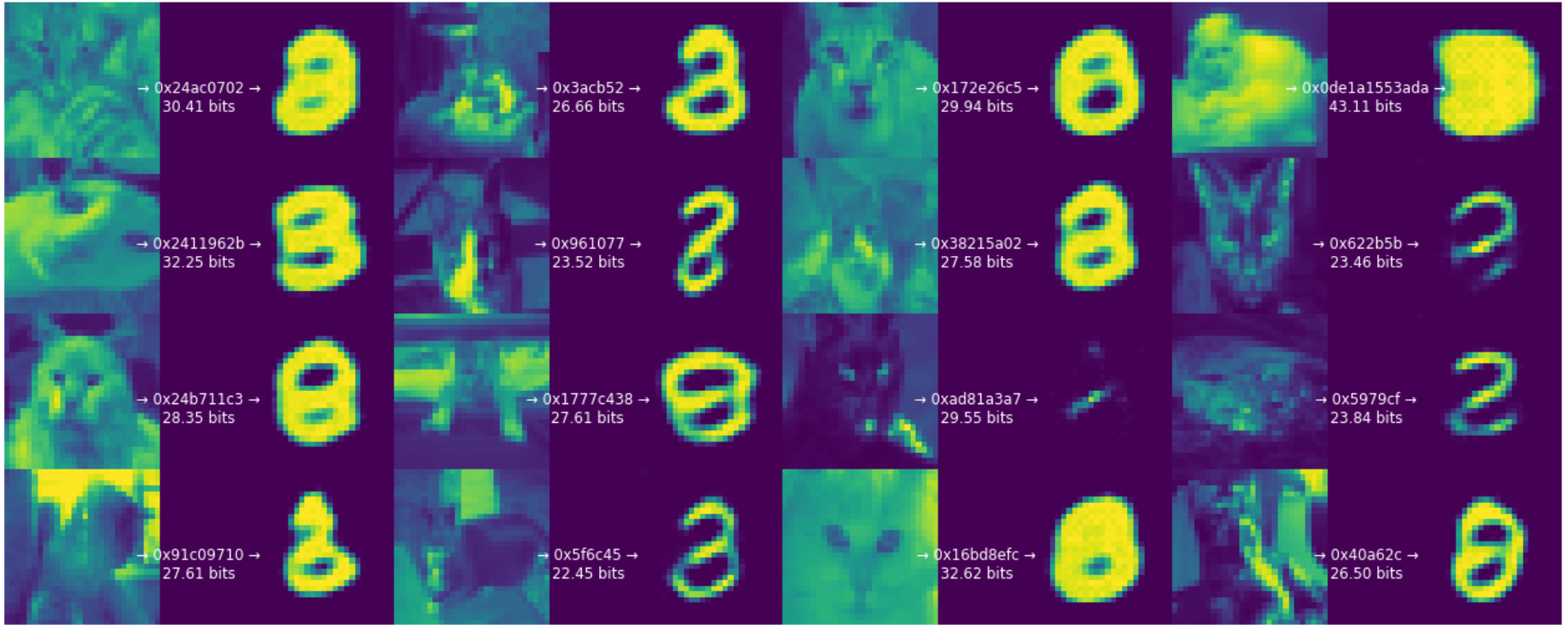
# MNIST compression (λ = 500)



# MNIST compression (λ = 300)

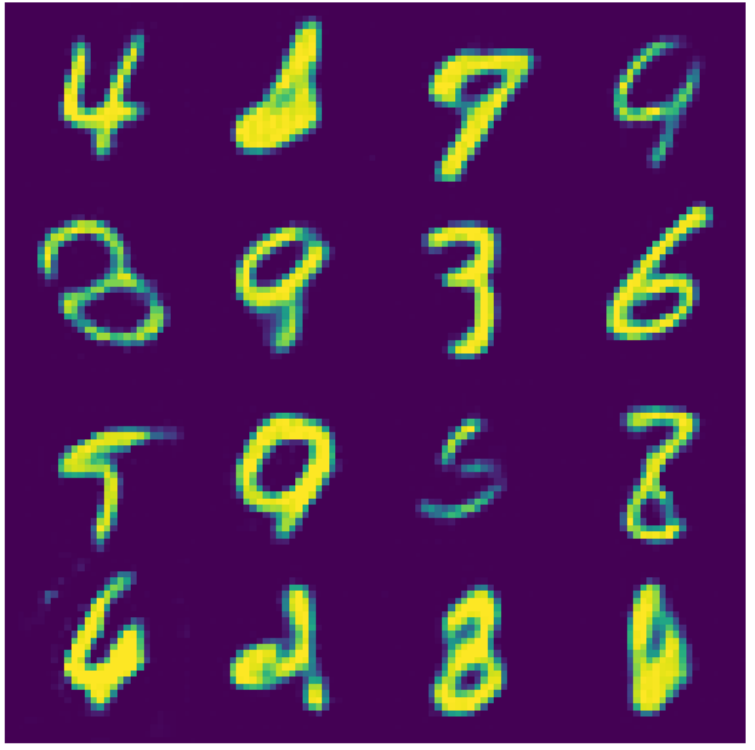


# MNIST compression



# MNIST compression

Results of image generation through the application of the learned synthesis transform on random binary strings.

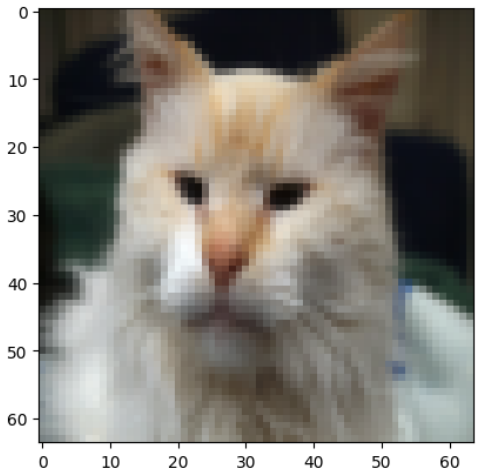


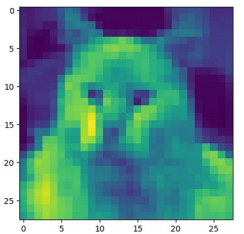
# Cats compression

**Dataset:** CATS
- training set cardinality: 72
- testing set cardinality: 33

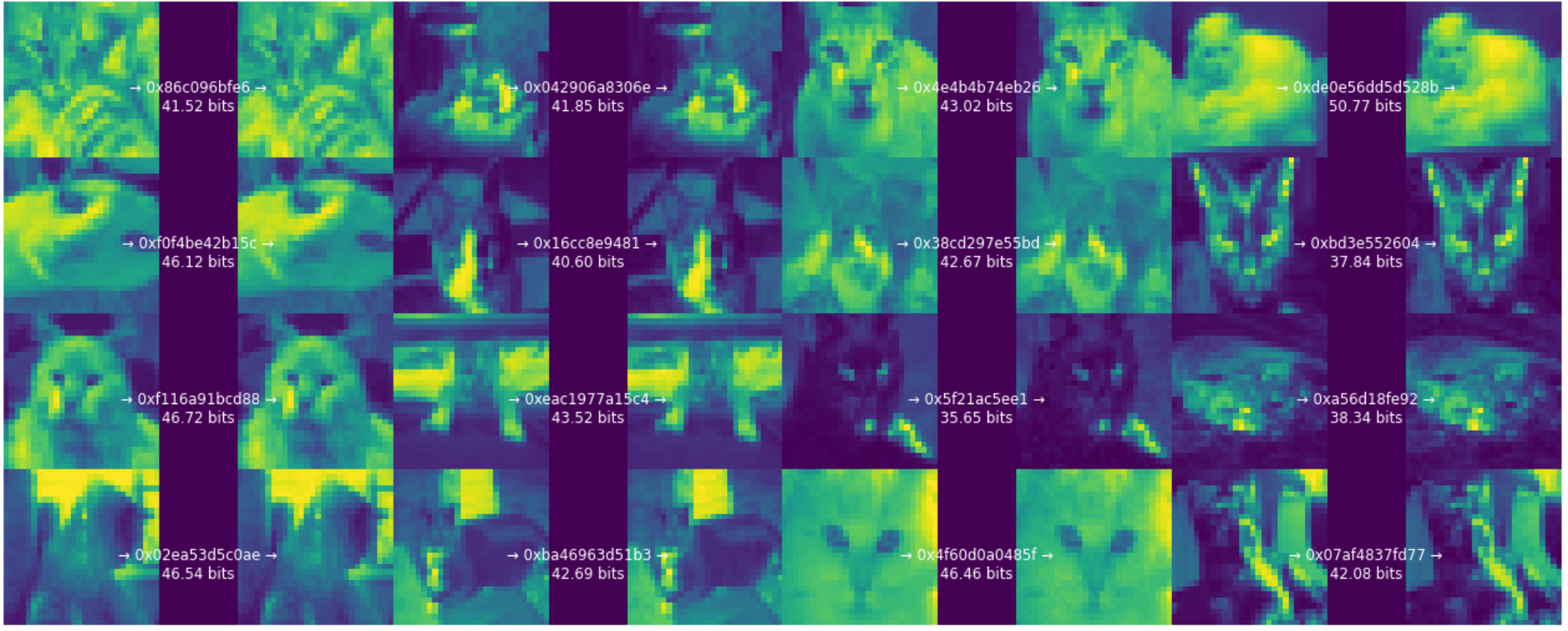
**Training**
- Distortion measure: MAE
- Optimizer: Adam
- Learning Rate: 0.001
- Epochs: 2000

- lambda: 20000
- latent dimension: 8

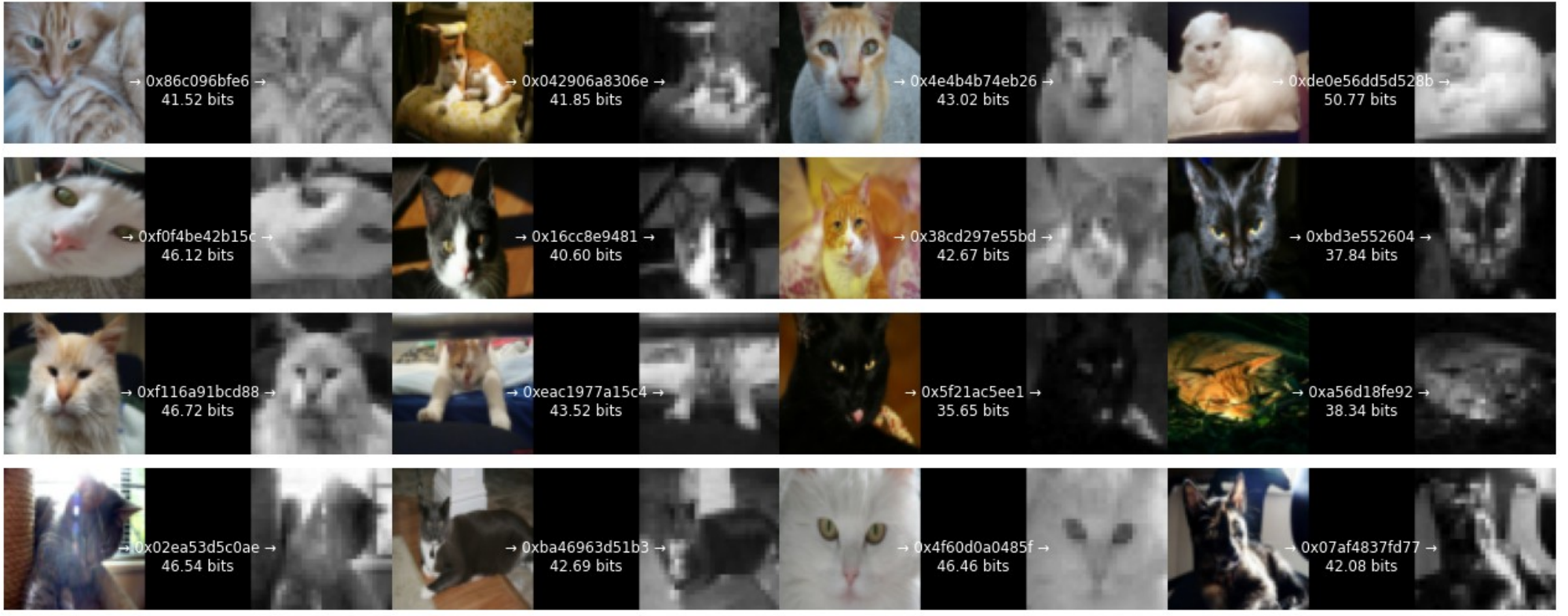


# Cats compression



# Cats compression



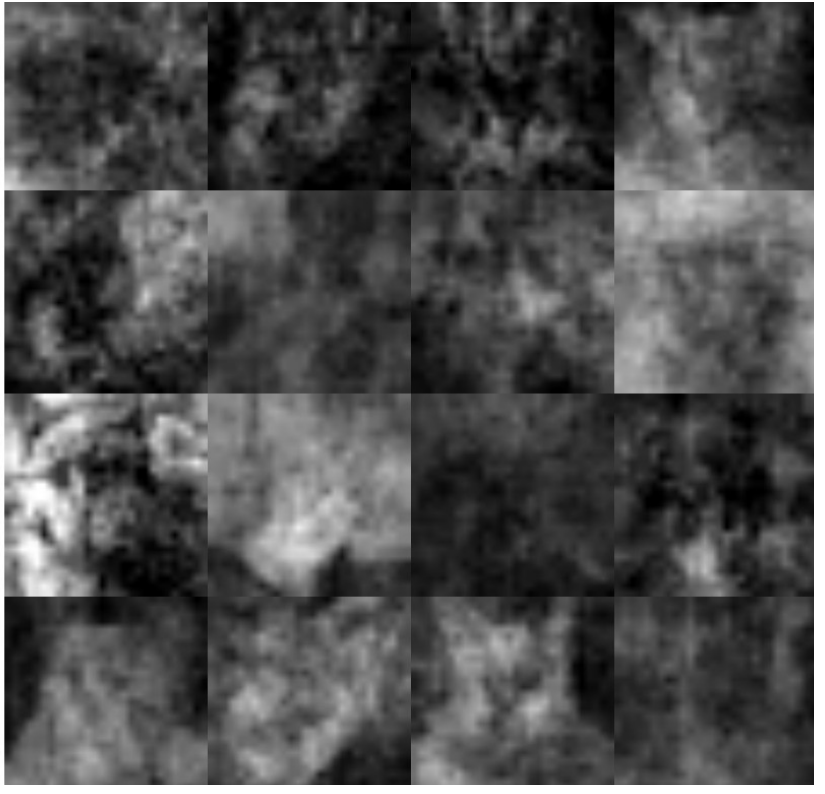
# Cats compression

Results of image generation through the application of the learned synthesis transform on random binary strings.

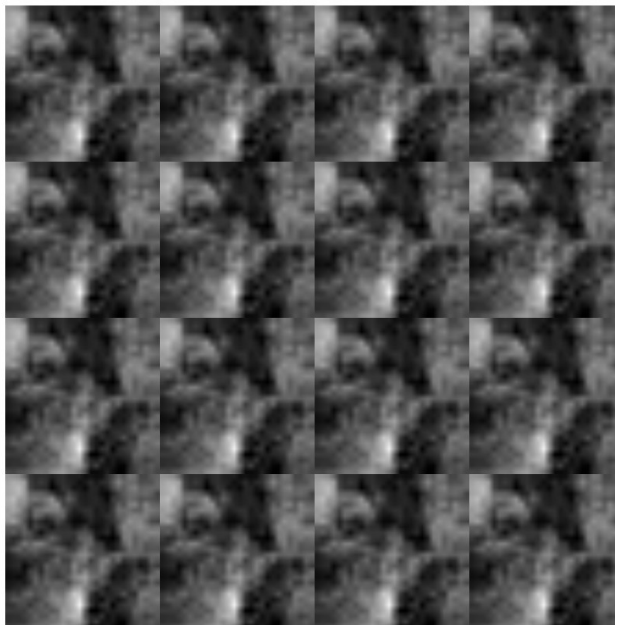
Poor results are probably mainly due to the size of the dataset.

With few epochs (< 1000) the resulting images are really blurry and  and noisy.

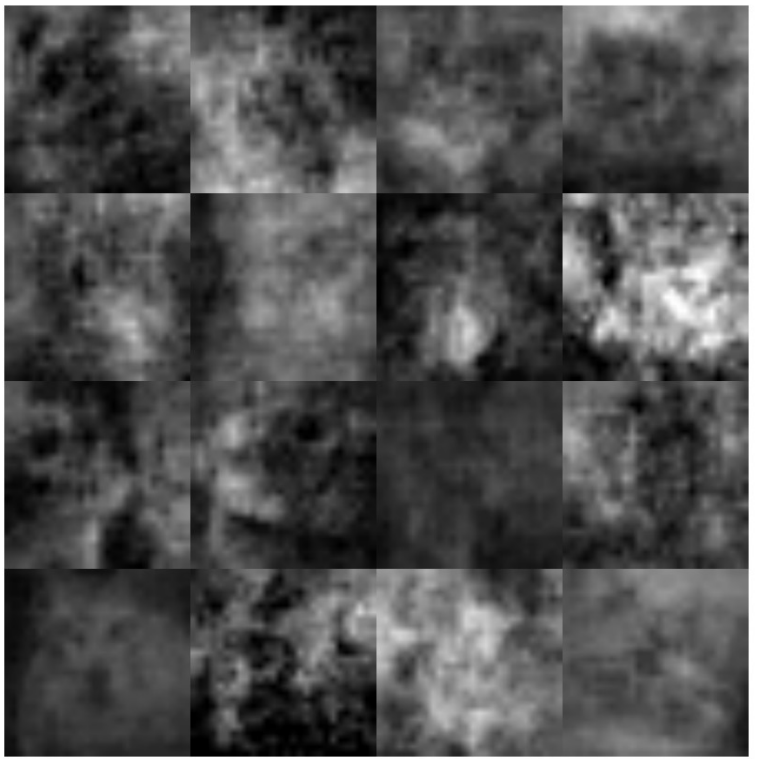
Generated cats images are likely to be a simple mixture of original images.

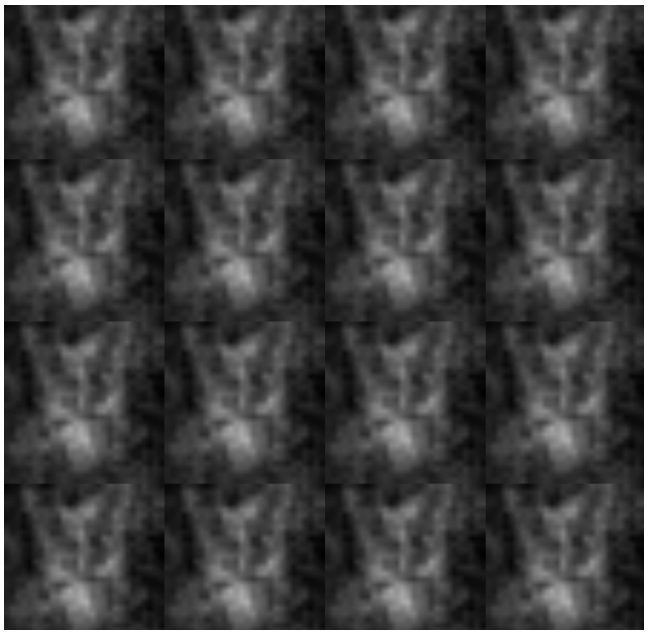


# Cats compression



Samu





Albi